



Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering Department

Fourth Year

Graduation Project Documentation

Gunnar Farneback Optical Flow Algorithm

Prepared By:

Name	Sec	BN	ID
Ahmed Mokhtar Mohamed	1	20	9220103

Submitted to:

Dr. Abdelrahman Abo Taleb

Contents

1	Gunnar Farneback Algorithm Formulation	2
1.1	Neighborhood Modeling	2
1.2	Constraint Equation and Solving for Motion	2
1.3	Calculating Coefficients	3
2	Algorithm Implementation (Python Pipeline)	4
2.1	Pipeline Execution Steps	4
3	Hardware Design Implementation	5
3.1	Pyramid Processing Module	5
3.2	Memory Contents	6
3.3	Optical Flow Processing Module	6
3.4	Specialized Processing Unit (SPU)	6
3.5	GF Calculations Module	7

List of Figures

1	Pyramid Processing Architecture	5
2	Optical Flow Processing Architecture	6
3	Specialized Processing Unit (SPU) Architecture	6
4	GF Calculations Core Architecture	7

1 Gunnar Farneback Algorithm Formulation

As part of the initial phase of this project, I studied the theoretical foundations of dense optical flow. The chosen method is the Gunnar Farneback (GF) algorithm. The entire algorithm is built on one core assumption, known as brightness constancy. This assumes that the brightness of an object's pixel ($f_1(x)$) remains approximately the same when it moves to a new position in the next frame ($f_2(x)$).

1.1 Neighborhood Modeling

The flow to find the motion vector d begins by modeling the brightness of a pixel's small neighborhood using a quadratic polynomial, where x is the vector (x, y) :

$$f(x) \sim x^T A x + b^T x + c$$

For the first frame, the model is defined as:

$$f_1(x) = x^T A_1 x + b_1^T x + c_1$$

Due to the brightness constancy assumption, the second frame is considered to be the first frame shifted by the motion vector d :

$$f_2(x) = f_1(x - d) = (x - d)^T A_1 (x - d) + b_1^T (x - d) + c_1$$

Expanding this equation yields:

$$= x^T A_1 x - x^T A_1 d - d^T A_1 x + d^T A_1 d + b_1^T x - b_1^T d + c_1$$

Since $d^T A_1 x = x^T A_1 d$ (as it is a scalar and matrix A is symmetric), we can rewrite the second frame as:

$$\begin{aligned} f_2(x) &= x^T A_1 x + (b_1 - 2A_1 d)^T x + c_1 - b_1^T d + d^T A_1 d \\ &= x^T A_2 x + b_2^T x + c_2 \end{aligned}$$

1.2 Constraint Equation and Solving for Motion

By comparing coefficients, we find a critical relationship:

$$A_2 = A_1$$

$$b_2 = b_1 - 2A_1 d$$

By rearranging this relationship and assigning average values for A and b , we derive the main constraint equation:

$$A(x)d(x) = \Delta b(x)$$

where $\Delta b(x) = -\frac{1}{2}(b_2(x) - b_1(x))$.

Because solving this for a single pixel is highly sensitive to noise, the algorithm solves for the d that best fits an entire neighborhood using a weighted least-squares minimization:

$$\operatorname{argmin}_{\Delta x \in I} \sum w(\Delta x) |A(x + \Delta x)d(x) - \Delta b(x + \Delta x)|^2$$

The solution to this minimization is:

$$d(x) = (\sum w A^T A)^{-1} \sum w A^T \Delta b$$

To simplify this calculation, we define matrices G and h :

$$G = \sum w A^T A$$

$$h = \sum w A^T \Delta b$$

yielding the final calculation for the motion vector:

$$d = G^{-1}h$$

Finally, this process is integrated with a multi-layer image pyramid to track both large and small scale movements accurately.

1.3 Calculating Coefficients

The values for A , b , and c for each pixel's neighborhood are not found directly. Instead, the algorithm solves for a set of six basis functions or "characterization coefficients," r_1 through r_6 . The quadratic polynomial $f(x)$ is fully expanded as:

$$f(x) = r_1 + r_2x + r_3y + r_4x^2 + r_5y^2 + r_6xy$$

This polynomial can be represented as a function of position using a basis matrix B and the coefficient vector r . For a single pixel, the basis row vector is defined as:

$$B = [1 \quad x \quad y \quad x^2 \quad y^2 \quad xy]$$

Finding these six r coefficients is formulated as a weighted least-squares problem. The algorithm solves for the r vector using a pseudo-inverse solution to minimize $\|Br - f\|_w$, where f is the intensity of a certain pixel and W is a weight.

The base solution is $r = (WB)^+Wf$. According to the general pseudo-inverse formula:

$$A^\dagger = (A^*A)^{-1}A^*$$

Applying this logic and expanding the weights such that $W^2 = W_aW_c$, the expression for r is rewritten as:

$$r = (B^*W_aW_cB)^+B^*W_aW_cf$$

The components of this calculation are defined as follows:

- r : The 6×1 Coefficient Vector.
- f : The 121×1 Image Intensity Vector.
- B : The 121×6 Basis Matrix.
- W_a : The 121×121 Applicability Weight Matrix, acting as a Gaussian filter.
- W_c : The 121×121 Certainty Weight Matrix.

Once the six r coefficients (r_1 through r_6) are found for a neighborhood, A , b , and c are built directly from them using the following relationships:

$$\begin{aligned} c &= r_1 \\ b &= \begin{pmatrix} r_2 \\ r_3 \end{pmatrix} \\ A &= \begin{pmatrix} r_4 & \frac{r_6}{2} \\ \frac{r_6}{2} & r_5 \end{pmatrix} \end{aligned}$$

With A and b known for both frames, the algorithm can calculate Δb , G , and h , and finally solve for the motion vector d .

2 Algorithm Implementation (Python Pipeline)

To bridge the gap between mathematical theory and hardware implementation, I developed a Python pipeline (`GF.py`). This implementation acts as a behavioral model, mimicking the constraints of an FPGA environment.

2.1 Pipeline Execution Steps

1. **Hardware ROM Initialization** (`init_hardware_roms`): Instead of calculating the pseudo-inverse in real-time for every pixel, the mathematical models (basis matrices and Gaussian weights) are pre-calculated to generate 2D filter kernels. These kernels act as Read-Only Memories (ROMs) in the hardware, providing the matrix used to extract the r coefficients via simple 2D convolution.
2. **Pyramid Generation**: The `costum_GF` function receives the `prev` and `curr` frames and scales them down incrementally to build an image pyramid. This allows the algorithm to detect coarse, large movements at the top level and refine them at the base level.
3. **Polynomial Expansion** (`compute_polynomial_expansion`): For a given pyramid level, the pre-calculated ROM filters are applied via `cv2.filter2D` to both the previous and current images. This outputs the r_1 to r_6 coefficients for both frames.
4. **Iterative Coordinate Mapping**: To iteratively refine the motion vector d , the algorithm enters an iteration loop. It begins by creating shifted coordinate maps (`map_x`, `map_y`) by adding the current estimation of d to the original spatial grid. This step determines exactly where in the second frame the algorithm needs to look to track the displaced pixels.
5. **Coefficient Fetching** (`integer_shift_coeffs`): To align the two frames, the algorithm must evaluate the second frame's base coefficients (A_{2_base} and b_{2_base}) at these new shifted coordinates. Instead of using complex floating-point bilinear interpolation the Python code simulates an FPGA BRAM read pointer offset. It rounds the shifted coordinates to the nearest integer and clips them to prevent out-of-bounds memory access, allowing the aligned coefficients (A_2 and b_2) to be fetched directly.

6. **Matrix Update and Cross-Correlation:** With the spatial alignment complete, the algorithm directly compares the "mapped" second frame (A_2, b_2) against the static first frame (A_1, b_1). It computes the averaged A matrix and the difference vector Δb . Using standard ALU operations, these are immediately multiplied to generate the raw matrix G and vector h , setting the stage for the spatial smoothing process.
7. **ALU Math and Box Filtering (`G_H_box_filter`):** The components of matrices G and vectors h are computed. To simulate hardware smoothing over a window, a separable 1D box filter is utilized, creating smoothed versions of G and h .
8. **Final Flow Calculation:** Using Cramer's rule on the smoothed G and h components, the differential flow vectors Δd_x and Δd_y are calculated and accumulated into the main flow field d . For the 2×2 system, the solution is formulated as:

$$d = G^{-1}h$$

$$\Delta d_x = \frac{G_{22}h_1 - G_{12}h_2}{G_{11}G_{22} - G_{12}^2}$$

$$\Delta d_y = \frac{G_{11}h_2 - G_{12}h_1}{G_{11}G_{22} - G_{12}^2}$$

$$d = d_{upperlayer} + \begin{pmatrix} \Delta d_x \\ \Delta d_y \end{pmatrix}$$

3 Hardware Design Implementation

The FPGA architecture is designed to execute the optical flow calculations in a highly parallel, streaming fashion. This section breaks down the individual hardware blocks I designed, their interfaces, and the data they process.

3.1 Pyramid Processing Module

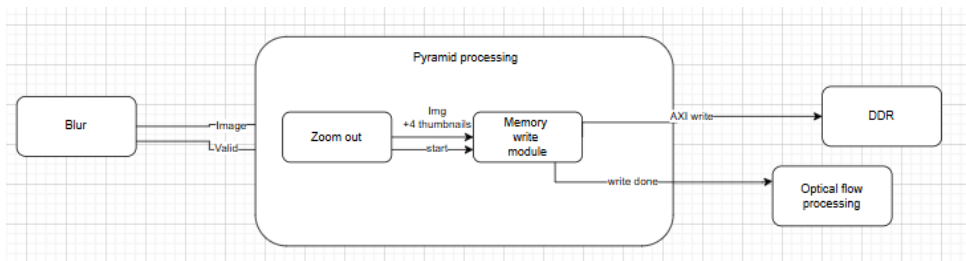


Figure 1: Pyramid Processing Architecture

Procedure: The pyramid processing module receives the blurred image from the pre-processing stage. It uses a Zoom Out block to down-sample the image iteratively with a ratio of 2:1, resulting in a five-layer pyramid.

Inputs: Valid, Pixel (from Blur module).

Outputs: AXI write commands to the PL DDR memory, Write done notification.

3.2 Memory Contents

The PL-side DDR memory acts as the central data hub for the pipeline. To support the concurrent processing of different pyramid levels without stalling, the memory maintains a rolling buffer that stores:

- **Image Pyramids:** Up to four consecutive image frames, including all their generated 5-layer pyramid thumbnails.
- **Optical Flow Data:** To conserve memory space, the system only retains and refreshes the calculated optical flow from the immediately preceding pyramid layer.

3.3 Optical Flow Processing Module

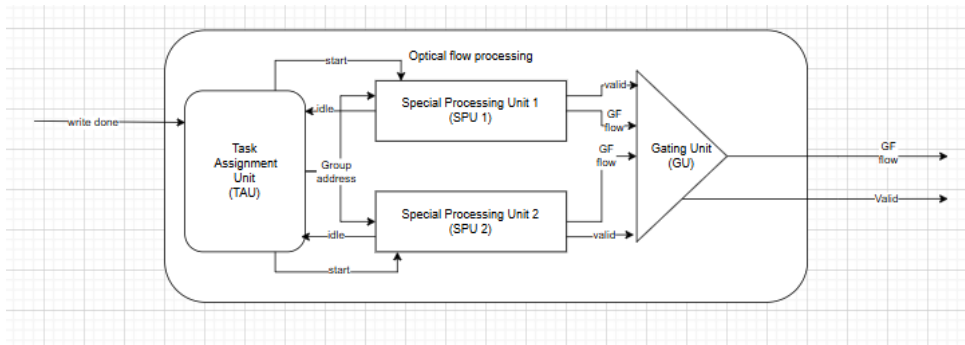


Figure 2: Optical Flow Processing Architecture

Procedure: This module manages the assignment and execution of the optical flow computations.

- **Task Assignment Unit (TAU):** To maintain high throughput, the system uses two Specialized Processing Units (SPUs). The TAU monitors the ‘idle’ status of both SPUs. When a write operation to memory completes, the TAU issues a ‘start’ command to the idle SPU, alongside the specific memory group address.
- **Gating Unit (GU):** Multiplexes the outputs from the two SPUs, selecting the active one to output the final GF flow and a valid signal.

Inputs: write done.

Outputs: GF flow, Valid.

3.4 Specialized Processing Unit (SPU)

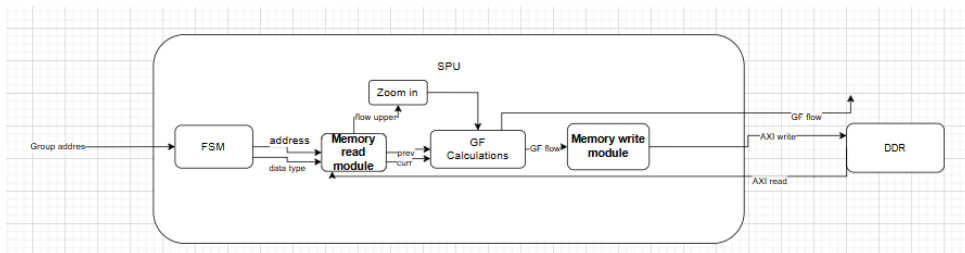


Figure 3: Specialized Processing Unit (SPU) Architecture

Procedure: The SPU handles the actual fetch-compute-store cycle for a given pyramid level.

- **FSM (Finite State Machine):** Calculates all memory addresses based on the group address and manages the state switching for the 5-layer pyramid.
- **Memory Read Module:** Fetches three data streams simultaneously via AXI: the previous frame, current frame, and the optical flow data from the upper pyramid layer (`flow_upper`).
- **Zoom In Module:** Uses bilinear interpolation to up-sample the `flow_upper` data to match the current layer's resolution.
- **Memory Write Module:** Sends the computed GF `flow` back to the DDR.

Inputs: Group address (from TAU), AXI read (from DDR).

Outputs: GF flow, AXI write (to DDR).

3.5 GF Calculations Module

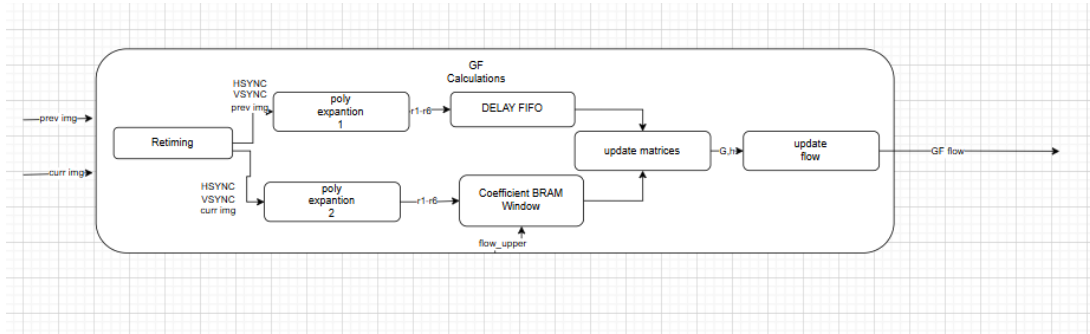


Figure 4: GF Calculations Core Architecture

Procedure: This is the arithmetic core of the algorithm within the SPU, operating heavily on Line Buffers and MAC units.

- **Retiming:** Synchronizes the incoming streams by generating HSYNC and VSYNC signals for memory modules.
- **Poly Exp 1 & 2:** Two parallel blocks that execute the spatial 2D filtering against the hardware ROMs to generate the basis coefficients r_1 through r_6 for both images.
- **DELAY FIFO & Coefficient BRAM Window:** The $r_1 - r_6$ coefficients for the previous frame are passed through a Delay FIFO to align them temporally. The current frame's coefficients pass into a BRAM Window. Using the `flow_upper` vector, integer addressing is used to shift the window and extract the aligned b_2 and A_2 matrices.
- **Update Matrices:** Computes the raw cross-correlation matrices G and vectors h .
- **Update Flow:** Applies box filtering to smooth G and h , and solves the system to output the final updated GF `flow` vector d .

Inputs: prev img, curr img, flow_upper.

Outputs: GF flow.